

renditeradar

Aggregator für Renditeobjekte, von der unsauberen Rohzeile bis zur Umkreissuche mit räumlichem Index

renditeradar liest Immobiliendaten ein, bereinigt sie, erkennt Duplikate, berechnet Rendite- und Preiskennzahlen und stellt sie als durchsuchbare, statisch erzeugte Seiten bereit. Gebaut als Arbeitsprobe für ein Mandat zu Immobilien-Aggregation, mit dem Ziel, zwei Dinge nachweisbar zu belegen: Next.js mit statischer Erzeugung und SEO, und PostgreSQL mit schnellen Filter- und Standortabfragen. Konzept, Datenmodell, Pipeline, Frontend und Betrieb in einer Hand.

6

Meilensteine von M0 bis M5, jeder mit Git-Tag und ausgeführtem Nachweis

≈3'500

Zeilen TypeScript, Python, SQL und CSS

51

Tests, 15 mit pytest und 36 mit Vitest

400

Objekte, idempotent geschrieben: der zweite Lauf ändert nichts

3

CI-Workflows, die die Projektregeln bei jedem Push erzwingen

100

Lighthouse in allen vier Kategorien auf der Detailseite

1 Kurzfassung

Eine Python-Pipeline erzeugt absichtlich unsaubere Rohdaten, normalisiert sie, erkennt Duplikate über einen Hash und schreibt sie mit berechneten Kennzahlen nach PostgreSQL. Eine Next.js-Anwendung zeigt die Objekte als gefilterte Liste und als statisch erzeugte Detailseiten mit Renditerechner, Vergleichsobjekten und Karte. Der gesamte Filterzustand steht in der Adresse, deshalb ist jede Ansicht teilbar und die Liste braucht keinen Zustand im Browser.

Der Kern in einem Satz

Jede Behauptung hat einen ausgeführten Nachweis: die Idempotenz durch einen zweiten Pipeline-Lauf, die Indexnutzung durch den Abfrageplan, das Schreibverbot durch einen abgewiesenen Schreibversuch mit dem öffentlichen Schlüssel.

Was das Projekt bewusst nicht enthält, gehört zur Aussage: kein Scraping realer Portale, keine Nutzerverwaltung, keine Zahlung. Der Umfang ist auf das begrenzt, was hier neu gezeigt werden soll, und das steht im Repository ausdrücklich statt stillschweigend weggelassen.

2 Ausgangslage und Ziel

Ausgeschrieben war ein Immobilien-Aggregator mit Scraping, Kennzahlen, Filtern, Renditerechner und einem Abo-Modell. Zwei Anforderungen daraus waren im vorhandenen Portfolio nicht belegt: Next.js mit serverseitiger und statischer Erzeugung für Suchmaschinen, und PostgreSQL mit schnellen Filter- und Standortabfragen. Alles Übrige, von Zahlung über Authentifizierung bis zum Betrieb gegen Anti-Bot-Massnahmen, war es bereits.

Daraus folgte der Zuschnitt: nicht ein kleines Abbild der ganzen Plattform, sondern genau die zwei fehlenden Belege, dafür vollständig und nachgewiesen. Angelegt auf einen Abend, mit einem definierten Abbruchpunkt nach der Bereitstellung. Die optionale Standortsuche kam erst danach.

Warum keine realen Portale ausgelesen werden

Ein öffentliches Repository mit Scraping-Code gegen ein reales Immobilienportal wäre kein Argument, sondern ein Risiko, rechtlich wie im Eindruck. Die Objekte sind deshalb generiert. Echt sind nur die 62 Gemeinden mit Postleitzahl, Kanton, Koordinaten und einem groben Preisniveau pro Quadratmeter. Für die Demonstration ist das gleichwertig: die Pipeline ist dieselbe, unabhängig davon, woher die Rohdaten kommen.

Meine Rolle

- **Konzept und Zuschnitt:** Anforderungen aus dem Inserat gegen das vorhandene Portfolio geprüft, Umfang auf die zwei fehlenden Belege begrenzt, Abbruchpunkt definiert.
- **Gestaltung:** Wireframe und UI-Mockup in zwei Stufen vor dem ersten Code, Design-Token aus dem Mockup direkt in die Konfiguration übernommen.
- **Datenmodell:** Schema mit Check-Constraints, zusammengesetzte Indizes, Row Level Security, später PostGIS mit erzeugter Spalte und räumlichem Index.
- **Pipeline:** Generator, Normalisierung, Duplikaterkennung über Hash, idempotenter Upsert, Tests.
- **Frontend:** Next.js mit App Router, Liste über Suchparameter, statische Detailseiten mit Erneuerung im Hintergrund, Metadaten, strukturierte Daten, Sitemap.
- **Qualität und Betrieb:** drei CI-Workflows, Bereitstellung auf Vercel, Messung mit Lighthouse.
- **Dokumentation:** vier Entscheid-Dokumente mit verworfenen Alternativen, vier Problem-Dokumente mit Ursache, offene Punkte je Meilenstein.

3 Funktionsumfang

- **Liste:** Filter nach Kanton, Objekttyp, Preisspanne, Mindestrendite und Umkreis um einen Ort; drei Sortierungen; Seitennavigation. Der gesamte Zustand steht in der Adresse.
- **Detailseite:** sechs Kennzahlen, Eckdaten, Renditerechner, die fünf nächstgelegenen Vergleichsobjekte mit Distanz, Karte.
- **Renditerechner:** fünf Eingaben als Schieberegler mit Zahlenfeld, vier Ergebnisse, bei jeder Eingabe neu berechnet. Grenzfälle geben "keine Angabe" statt einer unbrauchbaren Zahl.
- **Umkreissuche:** Ort und Radius aus einer Auswahl, aufgelöst über eine Datenbankfunktion mit räumlichem Index.
- **Suchmaschinen:** Titel und Beschreibung je Objekt, kanonische Adresse, strukturierte Daten für Objekt und Brotkrumen, Sitemap mit 401 Einträgen, robots mit Ausschluss der gefilterten Ansichten.
- **Pipeline:** Generator mit festem Startwert, Normalisierung, Duplikaterkennung, Upsert; ein Trockenlauf schreibt JSON statt in die Datenbank und braucht keine Zugangsdaten.

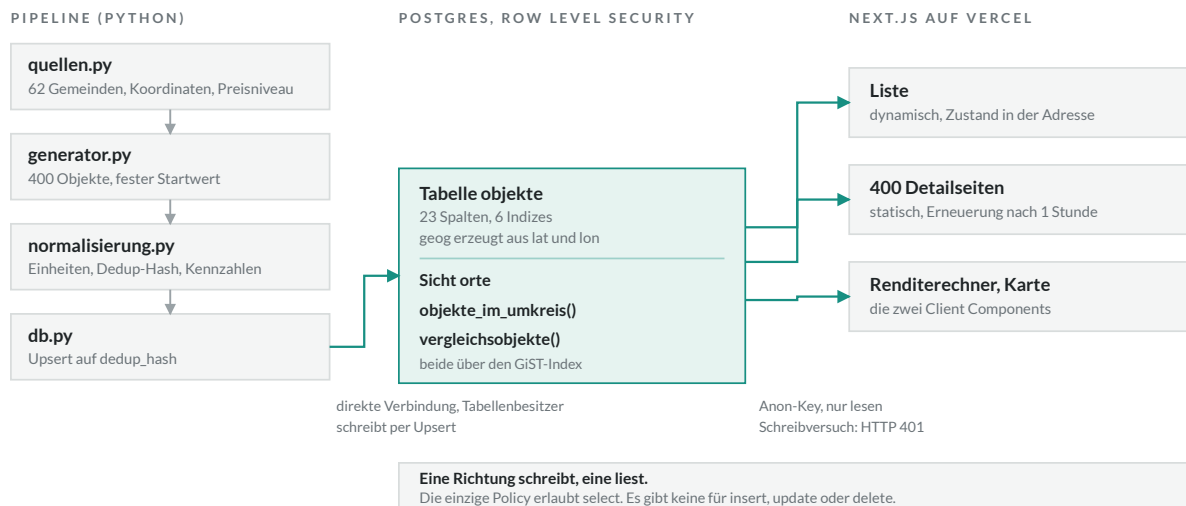
4 Architektur

Bereich	Technologie
Frontend	Next.js 16 mit App Router, React 19, TypeScript strict, Tailwind CSS v4, IBM Plex Sans über next/font, Leaflet für die Karte
Datenbank	PostgreSQL auf Supabase, Region Zürich, PostGIS 3.3, Row Level Security, Migrationen als SQL-Dateien
Pipeline	Python 3.13, psycopg 3, ruff für Format und Lint, pytest
Qualität	Vitest, ESLint, Prettier, tsc als Prüfung ohne Ausgabe, drei GitHub-Actions-Workflows
Betrieb	Vercel mit Wurzelverzeichnis <code>web</code> , statische Seiten mit Erneuerung im Hintergrund

Schichten

Schicht	Aufgabe
Pipeline	Erzeugt, normalisiert und schreibt die Objekte. Verbindet sich direkt mit Postgres als Tabellenbesitzer, Row Level Security gilt für sie nicht.
Postgres	Eine Tabelle mit 23 Spalten und 6 Indizes, eine Sicht über die Orte, zwei Funktionen für Umkreis und Nachbarn. Eine einzige Policy erlaubt <code>select</code> .
Next.js	Liest über die REST-Schnittstelle mit dem öffentlichen Schlüssel. Liste dynamisch, 400 Detailseiten statisch mit Erneuerung nach einer Stunde.
Browser	Zwei Client Components: der Renditerechner und die Karte. Alles Übrige wird auf dem Server gerendert.
Vercel, GitHub Actions	Bereitstellung aus <code>main</code> . Drei Workflows prüfen bei jedem Push Lint, Typen, Tests, Secrets in der Historie und die Rechtschreibregeln.

Systembild



Die Pipeline schreibt als Tabellenbesitzer über eine direkte Verbindung. Die Anwendung liest mit dem öffentlichen Schlüssel und kann nur lesen, weil die Datenbank es so festlegt und nicht der Anwendungscode.

Vier Grundsätze

- **Nachweisen statt behaupten.** Jedes Fertig-Kriterium wird ausgeführt und mit Befehl und Ausgabe festgehalten. Ein Meilenstein ohne Nachweis gilt als offen.
- **Der Server rechnet, der Browser zeigt.** Kennzahlen liegen als indizierte Spalten in der Datenbank. Nur der Renditerechner rechnet im Browser, mit Annahmen des Nutzers statt mit gespeicherten Werten.
- **Zustand gehört in die Adresse.** Filter, Sortierung und Seitenzahl stehen in den Suchparametern. Jede Ansicht ist teilbar, der Zurück-Knopf funktioniert, die Seite braucht keinen Zustand im Browser.
- **Was prüfbar ist, prüft die Maschine.** Rechtschreibung, Secrets in der Historie, fremde Inhalte, Lint, Typen und Tests laufen bei jedem Push. Regeln, die nur auf Disziplin beruhen, werden irgendwann übersehen.

Routen

Route	Zweck
Liste	Gefilterte Objekte, Zustand in der Adresse, bei jedem Aufruf gerendert
Detailseite	Kennzahlen, Eckdaten, Renditerechner, Vergleichsobjekte mit Karte; statisch erzeugt, stündlich erneuert
Sitemap, robots	401 Einträge; gefilterte Ansichten für Suchmaschinen ausgeschlossen
Nicht gefunden	Eigene Seite mit Rücksprung zur Liste, HTTP 404

5 Technische Kernprobleme

Sechs Stellen, an denen das Projekt über "Daten laden und anzeigen" hinausging. Alle sechs wurden erst durch eine Prüfung sichtbar, nicht durch das Lesen des Codes.

SICHERHEIT

Der öffentliche Schlüssel steht in jedem Browser

Problem

Die Anwendung liest mit einem Schlüssel, der im ausgelieferten Code steht und für jeden sichtbar ist. Eine Prüfung im Anwendungscode hilft nicht, weil die Schnittstelle direkt aufrufbar ist.

Lösung

Row Level Security mit genau einer Policy für `select`. Es gibt keine für `insert`, `update` oder `delete`. Was nicht erlaubt ist, ist verboten. Die Pipeline schreibt über eine direkte Verbindung als Tabellenbesitzer und ist davon nicht betroffen.

Warum es zählt

Sicherheit entsteht nicht dadurch, dass der Schlüssel geheim bleibt, sondern dadurch, dass die Datenbank selbst weiss, was mit ihm erlaubt ist. Der Beleg ist zwei Zeilen lang: der Leseversuch liefert Daten, der Schreibversuch wird mit HTTP 401 und Fehlercode 42501 abgewiesen.

DATEN

Derselbe Gegenstand, dreimal anders geschrieben

Problem

Ein Aggregator liest dieselbe Quelle täglich erneut. Ohne Wiedererkennung wächst die Tabelle jeden Tag um alles, was schon drinsteht. Und dieselbe Wohnung erscheint als "Aarburgerstrasse 12, 82 m²" und als "aarburgerstr. 12, 82.4", für einen Textvergleich zwei verschiedene Dinge.

Lösung

Ein Hash aus Postleitzahl, normalisierter Strasse und gerundeter Fläche, darauf ein Upsert. Die Normalisierung ist der eigentliche Teil: Kleinbuchstaben, Abkürzungen ausgeschrieben, Akzente entfernt, Leerraum gestrafft. Der Generator erzeugt die Rohdaten absichtlich unsauber, sonst hätte die Normalisierung nichts zu tun.

Warum es zählt

Der Nachweis ist der zweite Lauf: 400 neu und 0 aktualisiert beim ersten, 0 neu und 400 aktualisiert beim zweiten, Zeilenzahl unverändert. Die Grenze steht dokumentiert statt versteckt: zwei gleich grosse Wohnungen im selben Haus gelten als dasselbe Objekt. Bei echten Daten bräuchte der Hash die Stockwerkangabe.

DATENBANK

Ein räumlicher Index, der nicht benutzt wurde

Problem

Die Suche nach den fünf nächstgelegenen Objekten war als Verknüpfung der Tabelle mit sich selbst geschrieben. Sie lieferte die richtigen Ergebnisse in 0.63 Millisekunden. Der Abfrageplan zeigte aber einen sequenziellen Durchlauf über alle Zeilen mit anschliessendem Sortieren.

Lösung

Der Bezugspunkt steht jetzt als skalare Unterabfrage im Ausdruck statt als Partner der Verknüpfung. Damit wertet Postgres ihn einmal vorab aus, und er ist für den Planer eine Konstante. Erst dann kann der GiST-Index den Distanzoperator bedienen. Der Plan zeigt danach einen Index Scan, die Zeit fällt auf 0.16 Millisekunden.

Warum es zählt

Ein Index für die Suche nach dem nächsten Nachbarn setzt voraus, dass eine Seite des Vergleichs feststeht. Bei 400 Zeilen sehen beide Varianten sofort aus, und genau das ist die Falle: die eine wächst linear mit der Tabelle, die andere nicht. Der Unterschied steht im Plan, nicht auf der Uhr.

ARCHITEKTUR

Eine Filterleiste ohne Zustand im Browser

Problem

Fünf Filter, Sortierung und Seitenzahl sind der klassische Anlass, eine Seite zur Client Component zu machen. Damit wären Teilbarkeit, Zurück-Knopf und Erreichbarkeit für Suchmaschinen einzeln nachzubauen.

Lösung

Der Zustand steht in den Suchparametern. Die Filterleiste ist ein Formular mit GET-Methode, der Browser baut die Adresse selbst. Die aktiven Filter sind Links auf dieselbe Liste ohne einen Filter, das Blättern sind Links mit anderer Seitenzahl. Die Seite bleibt eine Server Component und funktioniert ohne JavaScript.

Warum es zählt

Bis zum Renditerechner enthielt die Anwendung keine einzige Client Component, danach genau zwei. Das ist kein Verzicht, sondern der Beleg, dass die Grenze zwischen Server und Browser bewusst gezogen wurde und nicht der Gewohnheit folgt.

ROBUSTHEIT

Drei Ergebnisse, die plausibel aussahen und falsch waren

Problem

Keiner der drei Fälle löste einen Fehler aus. Ein unbekannter Ort im Umkreisfilter lieferte alle 400 Objekte, während die Anzeige weiter "10 km um Atlantis" nannte. Ein unbekannter Slug lieferte fünf zufällige Vergleichsobjekte ohne Distanz. Und die Adresse `?preis_von=-5000` filterte nach einem Mindestpreis von 5'000, weil das Minuszeichen beim Bereinigen entfernt wurde.

Lösung

Ein Filter, der sich nicht auflösen lässt, führt jetzt zu null Treffern. Die Datenbankfunktion verlangt, dass es den Bezugspunkt gibt. Eingaben aus der Adresse werden geprüft statt bereinigt: was nicht der erwarteten Form entspricht, wird verworfen.

Warum es zählt

Ein Fehler, der als Fehler erscheint, wird behoben. Ein falsches Ergebnis, das wie ein Ergebnis aussieht, wird geglaubt. Bereinigen rät, was gemeint war, und rät manchmal das Gegenteil.

BETRIEB

Lokal grün, in der CI rot

Problem

Alle Prüfungen liefen auf dem Entwicklungsrechner ohne Befund. Der erste Lauf auf GitHub Actions scheiterte in allen drei Workflows, jeder aus einem anderen Grund. Der lehrreichste: der Typprüfer fand zwei Typen nicht, die Next.js erst beim Bauen erzeugt. Lokal war immer ein Build vorausgegangen.

Lösung

Jede Ursache zuerst lokal in einer leeren Umgebung reproduziert, dann behoben. Die Ursache der Typprüfung stand in den Meldungen des Laufs, nicht in einer Vermutung. Unterwegs fiel eine zweite, echte Abweichung auf, die aber nicht die Ursache war.

Warum es zählt

Eine Prüfung auf dem eigenen Rechner prüft dessen Zustand mit: erzeugte Dateien, ein zufällig passender Suchpfad, ein Build von vorhin. Erst die frische Umgebung zeigt, was das Repository tatsächlich enthält. Deshalb laufen die Projektregeln jetzt bei jedem Push.

6 Datenmodell

Eine Tabelle genügt. Die Kennzahlen sind gespeicherte Spalten und keine Berechnung beim Anzeigen, weil sonst weder nach Rendite sortiert noch nach einer Mindestrendite gefiltert werden könnte, ohne alle Zeilen zu laden.

Index	Spalten	Abfrage, die davon profitiert
unique	<code>dedup_hash</code>	Upsert der Pipeline
unique	<code>slug</code>	Detailseite
zusammengesetzt	<code>(kanton, typ, preis_chf)</code>	Listenfilter. Gleichheit vor Bereich, damit der Index auch bei nur einem Filter greift
absteigend	<code>bruttorendite</code>	Standardsortierung der Liste
GiST	<code>geog</code>	Umkreissuche und Suche nach dem nächsten Nachbarn

Die Lage als erzeugte Spalte

Die Spalte `geog` wird von der Datenbank aus `lat` und `lon` abgeleitet, bei jedem Schreiben neu. Sie kann deshalb nicht von den Koordinaten abweichen, und die Pipeline muss nichts von ihr wissen. Eine gepflegte Ableitung hätte an jeder schreibenden Stelle nachgeführt werden müssen, und die vergessene Stelle fällt erst auf, wenn eine Abfrage falsche Ergebnisse liefert.

Die doppelte Formel

Die Kennzahlen stehen zweimal: in der Pipeline für die gespeicherten Werte und im Renditerechner für die Annahmen des Nutzers. Beide Dateien tragen denselben Kommentarblock, beide haben Tests mit denselben Fällen. Das ist der offen benannte Preis dafür, dass nach Rendite sortiert werden kann.

Abweichung vom Plan	Grund
Primärschlüssel als fortlaufende Ganzzahl statt zufälliger Kennung	Zufällige Kennungen fragmentieren den Index. Die öffentliche Kennung ist ohnehin der Slug.
Geography-Spalte heisst <code>geog</code> statt <code>geom</code>	Übliche Unterscheidung zwischen den beiden räumlichen Typen. Der Plan sah <code>geom</code> vor.

7 Qualität und Nachweise

Kriterium	Nachweis
Pipeline ist idempotent	Zwei Läufe: 400 neu und 0 aktualisiert, danach 0 neu und 400 aktualisiert, Zeilenzahl unverändert
Listenfilter nutzt den Index	Index Scan auf dem zusammengesetzten Index, 0.143 ms
Umkreissuche nutzt den Index	Index Scan auf dem GiST-Index, 48 Treffer im Umkreis von 10 km um Olten, 7.8 ms
Nachbarsuche nutzt den Index	Sortierung über den Distanzoperator im Plan, 0.16 ms
Anwendung kann nicht schreiben	Schreibversuch mit dem öffentlichen Schlüssel: HTTP 401, Fehlercode 42501
Detailseiten sind statisch	X-VerceL-Cache: PRERENDER , Gültigkeit eine Stunde mit Erneuerung im Hintergrund
Zahlen auf der Seite stimmen	Sechs Filterkombinationen gegen die Datenbank verglichen, alle gleich
Grenzfälle im Rechner	Neun Tests: Eigenkapital 0 und 100 Prozent, Leerstand 100 Prozent, Kaufpreis 0, unsinnige Eingabe. Kein Fall ergibt eine unbrauchbare Zahl

Lighthouse gegen die laufende Seite

Seite	Leistung	Zugänglichkeit	Best Practices	SEO
Detailseite	100	100	100	100
Liste	97	100	100	100

Bei beiden Seiten liegt der Largest Contentful Paint unter 0.6 Sekunden, der Layout-Versatz bei null und die Blockierzeit bei null.

Was die Prüfungen gefunden haben

Die Qualitäts-Skills fanden acht Befunde, darunter zwei echte: die Listenseite hatte keine erste Überschrift und begann mit einer zweiten Ebene in den Objektkarten, und ein Zahlenfeld im Rechner entfernte den Fokusring ersatzlos. Lighthouse fand zwei weitere: ein Absatz stand als direktes Kind in einer Definitionsliste, und ein fehlendes Seitensymbol erzeugte bei jedem Aufruf einen Fehler in der Browser-Konsole. Alle behoben.

Ein Zwischenfall, der hierher gehört

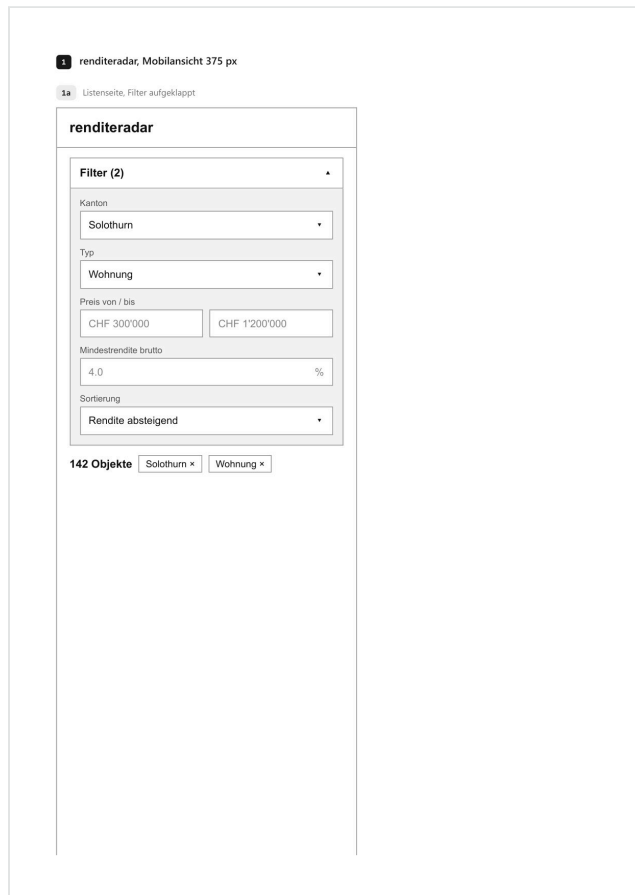
In der Anfangsphase schrieben zwei Sitzungen gleichzeitig im selben Repository. Dabei fehlte kurz ein Ignore-Eintrag, und 236 Dateien einer nicht frei lizenzierten Dokumentation gerieten in einen bereits gepushten Commit. Die Historie wurde neu geschrieben, bevor jemand das Repository geklont hatte, und der Vorfall steht in den offenen Punkten. Die Regel, nur benannte Dateien zu stagen, gilt seither und ist als Skill festgehalten. Dass es hier steht, ist Absicht.

Offene Punkte

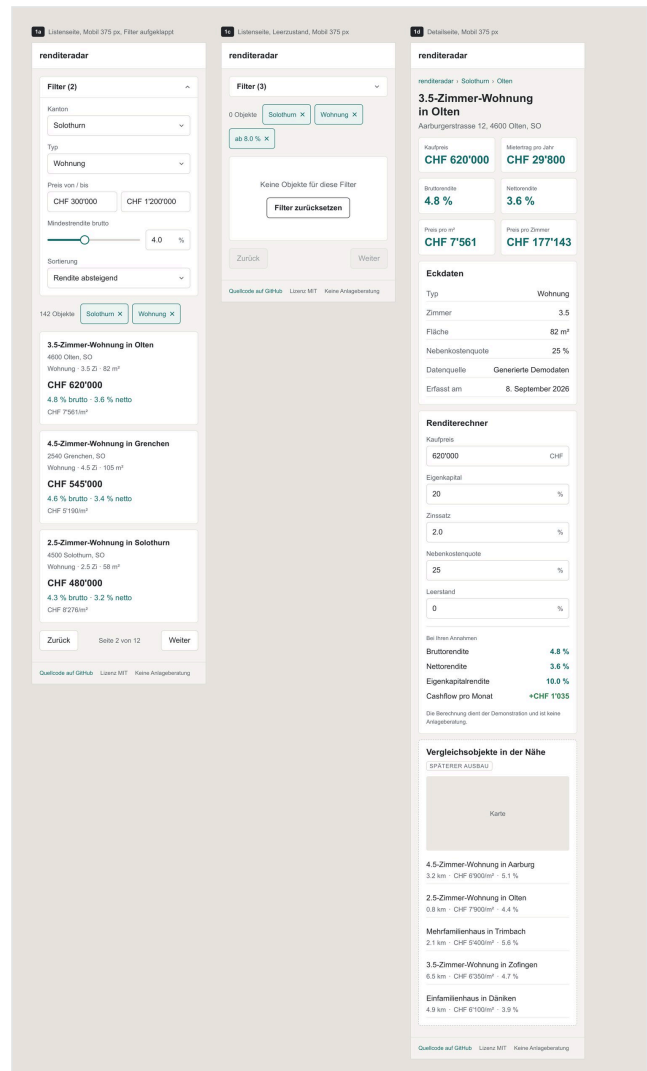
- Die Listenseite lässt sich nicht aus dem Vor- und Zurück-Zwischenspeicher des Browsers wiederherstellen. Das folgt aus dem Entwurf, weil der Zustand in der Adresse steht und die Seite deshalb bei jedem Aufruf gerendert wird.
- Die Sicht über die Orte mittelt die Koordinaten der Objekte eines Orts. Bei weit verstreuten Objekten ist dieser Mittelpunkt ungenau. Amtliche Ortskoordinaten wären besser.
- Der Tabellentyp im Frontend ist von Hand geschrieben. Die Erzeugung aus dem Schema braucht Docker oder ein Zugriffstoken, beides ist im Setup nicht vorausgesetzt.
- Die Abfragen lesen höchstens 1000 Zeilen auf einmal, weil die Schnittstelle nicht mehr liefert. Ab mehr Objekten müsste dort seitenweise gelesen werden.
- Die Karte ist nicht vollständig per Tastatur bedienbar. Alle Inhalte stehen zusätzlich als Liste darunter, die Karte ist Beiwerk.

8 Einblicke

Vor dem ersten Code entstanden zwei Gestaltungsstufen. Das Wireframe legt Aufbau und Gewichtung fest, in Graustufen und nur für Mobil. Das Mockup fügt Farbe, Schrift und die Desktop-Ansicht hinzu, ohne den Aufbau zu verändern. Die Farbwerte und Schriftgrößen aus dem Mockup wurden direkt in die Konfiguration der Anwendung übernommen.



Stufe 1, Wireframe. Liste, Leerzustand und Detailseite. Graustufen, keine Schriftwahl, Rechtecke mit Beschriftung statt fertiger Bedienelemente.



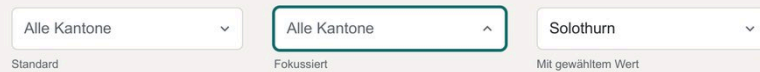
Stufe 2, Mockup. Dieselben Ansichten mit Farbe und Typografie. Der Aufbau bleibt unverändert.

11 Komponenten mit Zuständen

Filter-Chip



Auswahlfeld



Zahlenfeld



Schieberegler mit Zahlenfeld



Objektkarte



Kennzahl-Kachel



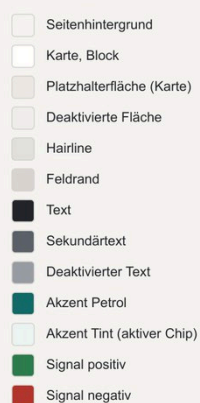
Seitennavigation



Knopf



Farbwerte



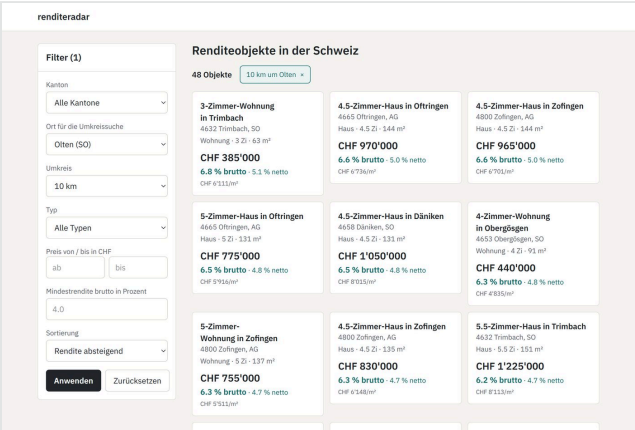
Schriftgrößen

Kennzahl Detailseite	28 px, 600
Kennzahl Detailseite Mobil	24 px, 600
Seitentitel	24 px, 600
Kaufpreis Karte	20 px, 600
Abschnittstitel, Wortmarke	18 px, 600
Fliesstext, Felder	16 px, 400
Sekundärtext, Labels, Chips	14 px, 400
Kleintext, Fussnoten	12 px, 400
IBM Plex Sans, 400 und 600. Tabellenziffern für alle Zahlen. Zeilenhöhe 1.5 im Fliesstext, 1.2 bei Kennzahlen und Titeln. Ecken 6 px. Basisraster 4 px.	

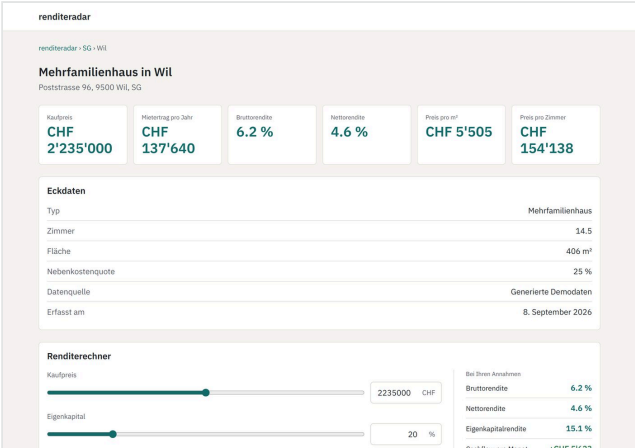


Komponenten mit Zuständen. Jedes Bedienelement in allen Zuständen, dazu die Farbwerte als Hex und die Schriftgrößen. Dieses Blatt wurde direkt zur Konfiguration der Design-Token.

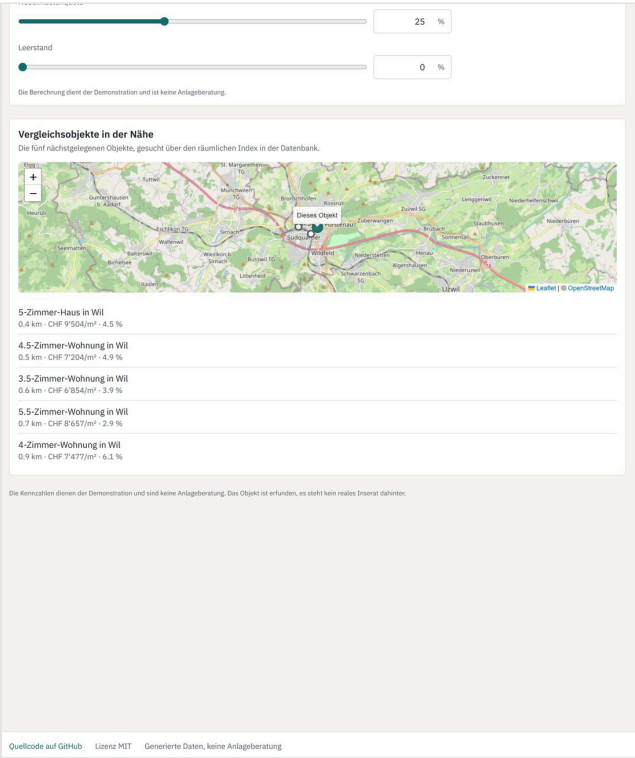
Die umgesetzte Anwendung



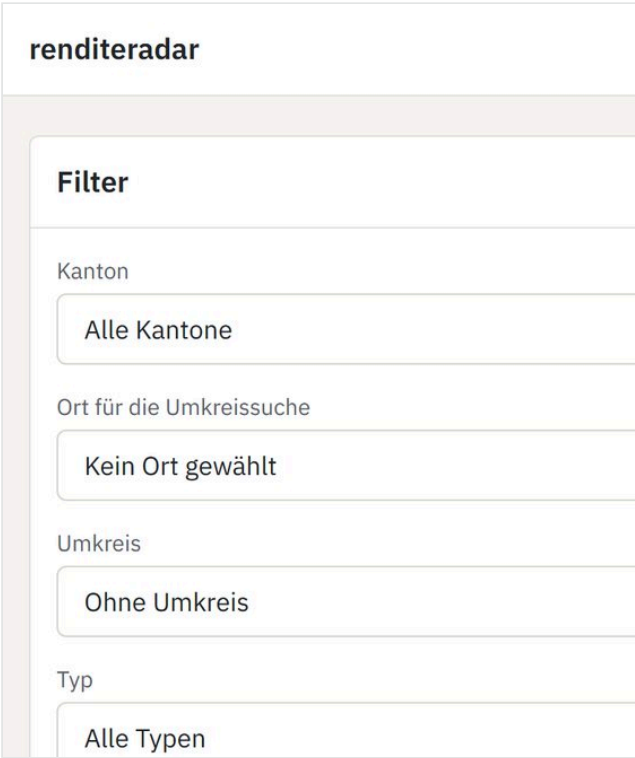
Liste mit Umkreissuche. 48 Objekte im Umkreis von 10 km um Olten. Der aktive Filter erscheint als entfernbarer Chip, die Adresse enthält den ganzen Zustand.



Detailseite. Sechs Kennzahlen, Eckdaten und der Renditerechner. Beim Bauen statisch erzeugt und aus dem Zwischenspeicher ausgeliefert.



Vergleichsobjekte und Karte. Die fünf nächstgelegenen Objekte, gesucht über den räumlichen Index. Die Liste steht unabhängig von der Karte.



Mobil. Eine Spalte, Filter aufklappbar, Bedienelemente mindestens 44 Pixel hoch. Null Treffer sind ein Ergebnis mit Rücksetzknopf, kein Fehler.

9 Was das Projekt zeigt

Fähigkeit	Wo sie sichtbar wird
Next.js mit App Router	400 statisch erzeugte Detailseiten mit Erneuerung im Hintergrund, dynamische Liste über Suchparameter, Metadaten und strukturierte Daten je Objekt
Server und Client trennen	Zwei Client Components im ganzen Projekt. Filter, Chips und Blättern laufen über Adresse und Formular und funktionieren ohne JavaScript
PostgreSQL jenseits von SELECT	Zusammengesetzte Indizes in der richtigen Spaltenreihenfolge, Row Level Security als Schreibschutz, erzeugte Spalte, Abfragepläne gelesen statt Laufzeiten geraten
Räumliche Abfragen	PostGIS mit GiST-Index, Umkreissuche über <code>ST_DWithin</code> , Nachbarsuche über den Distanzoperator, beide mit belegter Indexnutzung
Datenaufbereitung	Normalisierung unsauberer Rohdaten, Duplikaterkennung über Hash, idempotenter Upsert mit ausgeführtem Nachweis
Suchmaschinen und Zugänglichkeit	Lighthouse 100 in allen vier Kategorien auf der Detailseite, Sprungmarke, Live-Region, Fokusringe, Tap-Ziele
Qualität automatisieren	Drei CI-Workflows erzwingen Rechtschreibung, Secrets-Prüfung, Lint, Typen, Tests und die Vollständigkeit der Meilenstein-Dokumentation
Ehrliche Dokumentation	Vier Entscheide mit verworfenen Alternativen, vier Problem-Dokumente mit Ursache, offene Punkte je Meilenstein, ein Zwischenfall offen benannt

Stand	September 2026 · alle Meilensteine abgeschlossen, Demo öffentlich erreichbar
Kontakt	Gabriel Suter · kontakt@gabrielsuter.dev · gabrielsuter.dev
Quellcode	github.com/SuterGabriel/renditeradar, öffentlich, MIT-Lizenz, Commit-Verlauf mit sechs Meilenstein-Tags
Live	renditeradar-wine.vercel.app
Haftungsausschluss	Alle Objekte sind generiert, es steht kein reales Inserat dahinter. Die Renditeberechnungen dienen der Demonstration und sind keine Anlageberatung.